AI-powered Virtual Humans

LLM-Powered Cognitive Architectures



Module 2 - Chapter 01
Agentic Frameworks and LangGraph

Why Agentic Frameworks? [5] [6]

Tools and memory

Call external functions. Keep the conversation context.

Communication

Coordinate steps, agents and the people involved.

Reliability

Checks, guardrails and recovery after failures.

Ready-made abstractions instead of building every component from scratch.

Agentic Frameworks [5] [6]

LangGraph [1]	CrewAI [2]	AutoGen [3]	Semantic Kernel [4]
State and graph	Roles and tasks	Messages and teams	Plugins and functions
Explicit loops and branching	Coordinated agent teams	Agent conversations	AI in existing applications

Capabilities overlap; LangGraph makes state and control flow explicit.

LangChain Components [10] [5]

langchain

Model

One interface for many providers.

Prompt

Reusable templates for model input.

Tool

Functions the model can call.

Memory

Short-term and long-term context.

Agent

A model-driven decision loop.

Output parser

Raw text to structured data.

Runnables compose with LCEL: prompt | model | parser, as in Module 1.

LangGraph Building Blocks [1] [5]

langgraph

Graph

The flow of steps: directed, possibly cyclic.

State

Shared data that every node reads and updates.

Nodes

Units of work: Python code, model or tool calls.

Edges

Sequential, conditional or cyclic transitions between nodes.

Checkpointer

Saves the state between runs, so a conversation can resume.

Today: graph, state, nodes and edges. Chapter 02: the checkpointer.

Why LangGraph for a Virtual Human [1] [5] [11]

Strengths

Explicit branches, loops and fallbacks.
State shared across conversation turns.
Checkpoints to resume a conversation.
Human-in-the-loop pauses and streaming.

Costs

More design work.
A steeper learning curve.
Too much for simple prototypes.

Tracing and monitoring: LangSmith [11], in the same ecosystem.

Conversation History and State [1] [7]

Your list exercise

Keep human and AI messages.
Send the history on each call.

Graph state

A dictionary holds the data used by the workflow.
Messages can be one of its fields.

The history can still be a list inside the state.

LangChain and LangGraph [1]

LangChain components

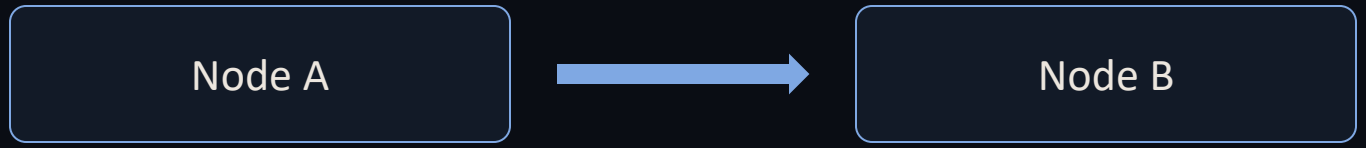
Prompts, messages, models and structured responses.

LangGraph execution

State, execution order, branches and loops.

A graph node can call a chain or run ordinary Python code.

State, Nodes and Directed Edges [9]



A directed edge runs from A to B.

State

Data available to the nodes.

Nodes

Functions that do the work.

Edges

Where execution goes next.

State: a Dictionary [9]

```
class VisitorState(TypedDict):
    visitor: str
    greeting: str
    opening_hours: str
    answer: str
```

Input
{"visitor": "Lea"}

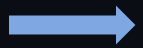
Nodes fill the remaining fields as the graph runs.

TypedDict describes keys and types; it does not validate values.

Nodes Return State Updates [9]

```python
def greet(state: VisitorState):
    return {"greeting": f"Hello {state['visitor']}!"}
```

visitor: "Lea"

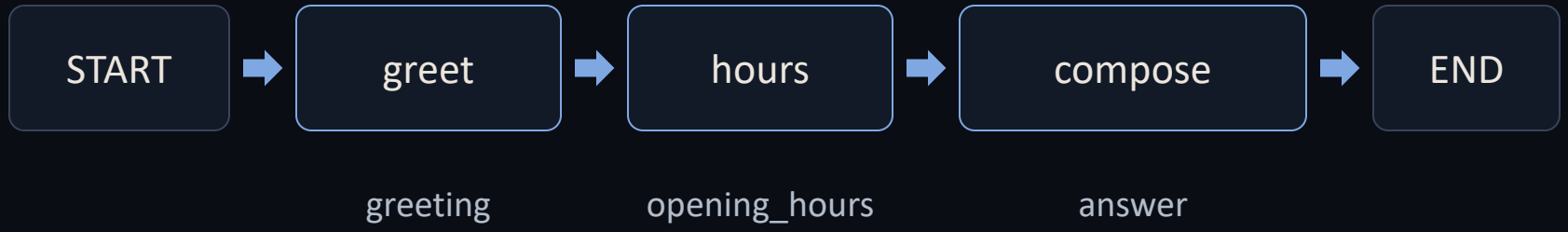


visitor: "Lea"
greeting: "Hello Lea!"

Return changed fields. Other fields remain in the state.

A Sequential Graph [9]

Greet the visitor, get opening hours, then compose the answer.



Hello Lea! The museum opens at 10:00.

Build and Compile [8] [9]

```
builder = StateGraph(VisitorState)
builder.add_node("greet", greet)
builder.add_node("hours", hours)
builder.add_node("compose", compose)

builder.add_edge(START, "greet")
builder.add_edge("greet", "hours")
builder.add_edge("hours", "compose")
builder.add_edge("compose", END)

graph = builder.compile()
```

Invoke the Graph [9]

```
result = graph.invoke({"visitor": "Lea"})
print(result["answer"])
```

Input dictionary

The values supplied at the start of this run.

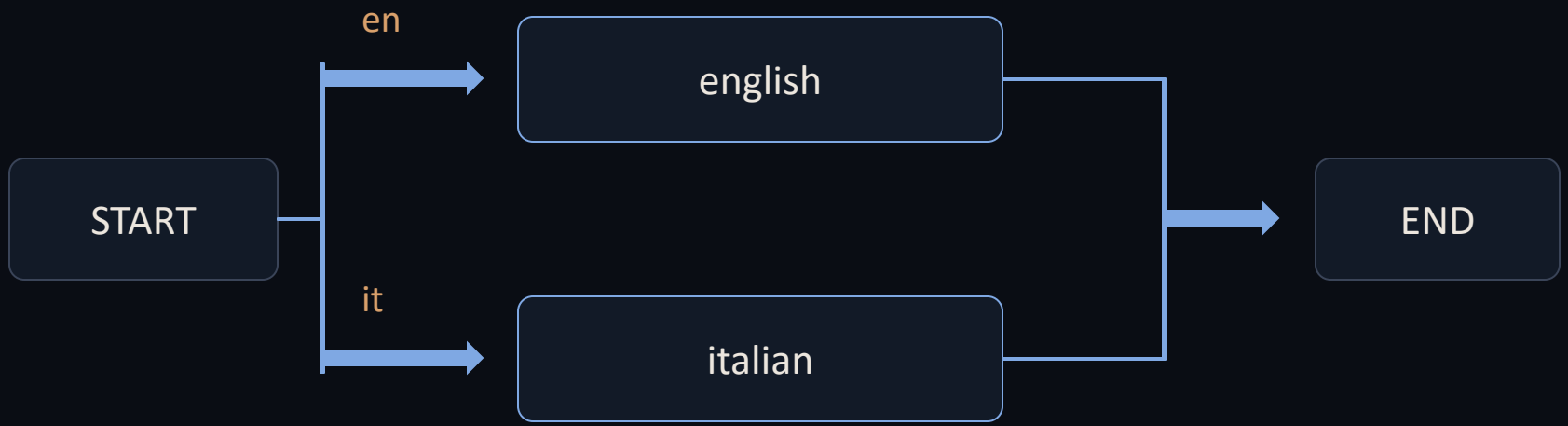
Output dictionary

The final state, including the answer.

Open 01_langgraph.ipynb: 1. A sequential graph

Conditional Paths [9]

Choose one greeting using the language in the input.



Routing from State [9]

```python
def select_language(state: LanguageState):
    return state["language"]

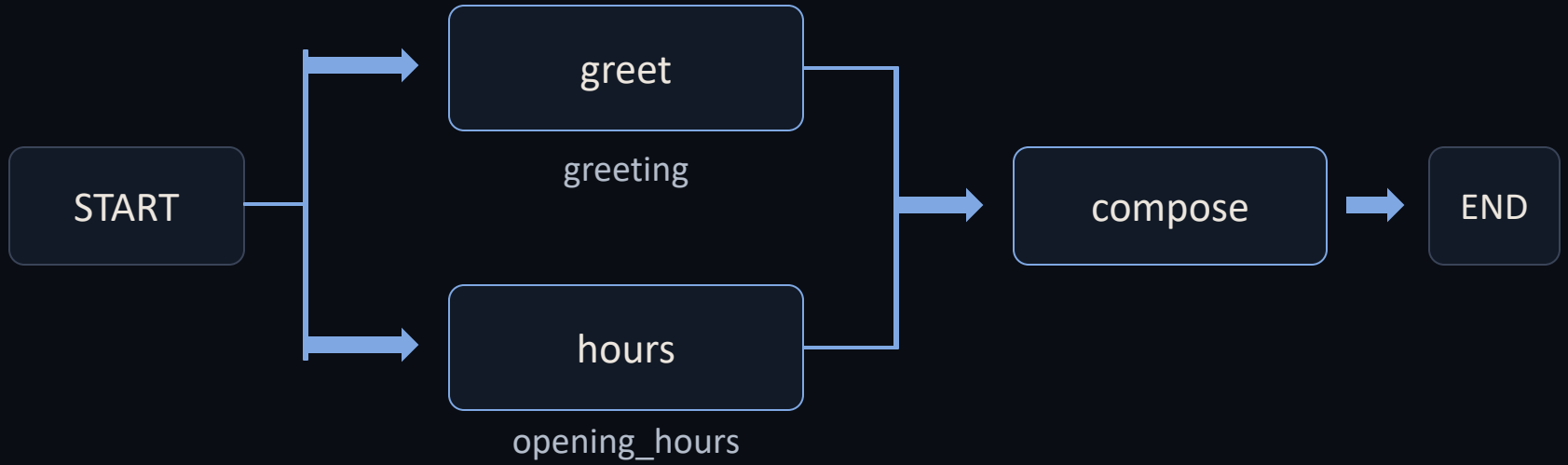
builder.add_conditional_edges(
    START, select_language,
    {"en": "english", "it": "italian"}
)
```

Try English and Italian. Then add a French greeting.

Open 01_langgraph.ipynb: 2. A conditional path

Parallel Paths [9]

Independent nodes can run in the same step.



Split and Join [9]

```
builder.add_edge(START, "greet")
builder.add_edge(START, "hours")
builder.add_edge(["greet", "hours"], "compose")
builder.add_edge("compose", END)
```

The list makes compose wait for both nodes.

Each parallel branch writes a different state key.

Open 01_langgraph.ipynb: 3. Parallel paths

An LLM Inside a Node

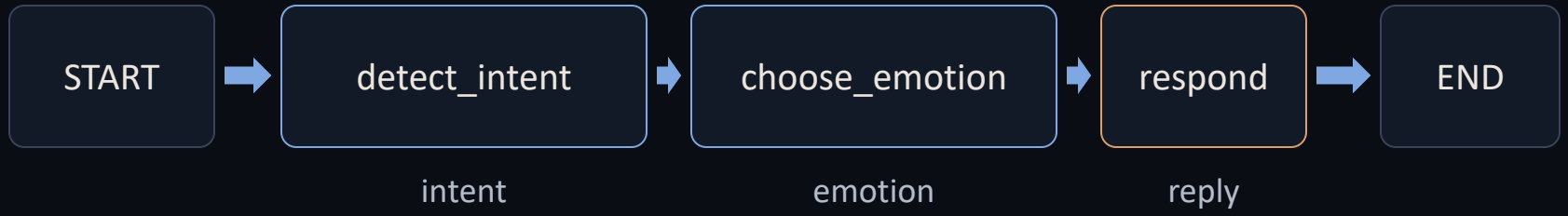


Together: write the two nodes and connect them.

The list stays outside the graph: we pass it in and append the reply.

Open 01_langgraph.ipynb: 4. Together: an LLM inside the graph

Exercise: A Dialogue Manager



Intent

GREETING, QUESTION, COMPLAINT, THANKS or GOODBYE

Emotion

From the avatar catalog: the reply must express it.

Open 01_langgraph.ipynb: 5. Exercise: a dialogue manager

Next: Conversation Memory [7] [9]

Today

The list lives outside the graph.
We pass it in on every call.

Chapter 02

Messages in state.
A reducer appends them.
A checkpointer saves them.

Next chapter: the graph remembers the conversation by itself.

References

[1] LangGraph

Stateful workflows and explicit control flow

[2] CrewAI

Role-based teams, tasks and flows

[3] AutoGen

Conversational agents and message-driven systems

[4] Semantic Kernel

Plugins and integration with existing applications

[5] Proglio (2026)

Agentic frameworks, thesis section 2.5

[6] Derouiche et al. (2025)

Agentic AI frameworks: comparative survey

[7] LangGraph persistence

State snapshots and conversation continuity

[8] LangGraph quickstart

Build and compile a graph

References (continued)

[9] LangGraph Graph API

State, nodes, edges and parallel execution

[10] LangChain

Components for LLM applications

[11] LangSmith

Tracing and observability for LLM applications